

ARC-AGI-2: Explicit CPU Rules, Qwen 3, and Parameter-Efficient Adaptation with Low-Rank Adapters

Umar Tabbsum
Independent Researcher
Dublin, Ohio, USA

Abstract

The Abstraction and Reasoning Corpus (ARC) evaluates whether an AI system can infer a transformation from a few visual-grid demonstrations and apply it to an unseen input. This report documents a preliminary ARC-AGI-2 project that compares deterministic CPU rule verification with a neural experiment on Qwen 3 1.7B. The recorded exploratory analysis covers 1,000 training tasks and 3,232 training input–output pairs; 320 tasks (32%) contain at least one dimension-changing demonstration. Two selected deterministic rules reproduced seven of seven checked demonstrations. In the recorded neural run, validation loss fell from 0.6996 to 0.4407 (a 37.0% relative reduction, i.e. per-token perplexity from about 2.01 to 1.55), yet a separate 32-case evaluation recorded 32 syntactically valid grids but zero exact matches. The two results use different scopes and are not a controlled head-to-head comparison. We give a linear-algebraic account of grid representations, of the transformer and Qwen 3 architecture, and of Low-Rank Adaptation (LoRA), with an explicit parameter budget for the 1.7B configuration; we survey alternative model families (Gemma, Llama, Mistral); we place the work in the context of ARC Prize 2025 and test-time training; and we define the statistical protocol needed for a rigorous comparison. The current evidence does not establish an official leaderboard score, broad generalization, novelty, or achievement of the 85% target.

CCS Concepts

• **Computing methodologies** → **Knowledge representation and reasoning**; *Machine learning*.

Keywords

ARC-AGI-2, abstract reasoning, Qwen 3, Gemma, transformers, LoRA, test-time training, rule-based AI, generalization

1 Introduction

ARC tasks provide a small number of input–output grid examples. A solver must infer a rule that explains the demonstrations and apply it to a new grid. The challenge is not merely to produce a plausible output: the predicted dimensions and every cell must be correct.

The ARC framework emphasizes generalization and the efficiency with which a system acquires new skills, rather than memorization of familiar examples [7]. ARC-AGI-2 retains this focus and targets symbolic interpretation, compositional reasoning, and contextual rule application [3, 8]. The ARC Prize 2025 technical report identifies the *refinement loop* (a per-task iterative optimization guided by a feedback signal) as the defining idea of that year’s progress [9].

This project investigates three complementary ideas:

- (1) **Exploratory data analysis:** characterize dimensions, colors, and grid transformations (Section 3).
- (2) **Explicit CPU rules:** implement and verify interpretable candidate transformations (Section 4.1).
- (3) **Neural adaptation:** examine whether adapting Qwen 3 with a low-rank adapter improves the generation of structured grid outputs (Sections 4.2–4.4).

Contributions. (i) A reproducible record of dataset statistics, rule verification, and a neural run, with explicit statements of what each measurement does and does not show. (ii) A matrix formulation of ARC grids, symmetries, candidate rules, attention, and LoRA, including a derived parameter and memory budget for Qwen 3 1.7B. (iii) A comparison of candidate model families and a literature-grounded experimental protocol with confidence intervals and paired tests.

Organization. Section 2 reviews related work; Section 3 describes the dataset and its figures; Section 4 describes the rules, the Qwen 3 model, the transformer, and LoRA; Section 5 compares model families; Section 6 defines metrics; Section 7 reports results with their figures; Sections 8–12 discuss, plan, and assess; the appendix indexes the raw artifacts, and the references close the report.

The report is a preliminary experimental record, not a claim of a competitive solution. Its purpose is to distinguish what the current measurements demonstrate from what still requires controlled testing.

2 Background and Related Work

2.1 The ARC-AGI-2 benchmark

ARC-AGI-2 contains 1,000 public training tasks and three evaluation sets of 120 tasks each (public, semi-private, and private); tasks use integer colors 0–9 on grids of at most 30×30 [8]. In the controlled human study, more than 400 participants attempted candidate tasks, and every retained evaluation task was solved by at least two people in two attempts or fewer [8]. Tasks that were susceptible to brute-force program search were removed, and new tasks were added to probe compositional and contextual reasoning. Scoring allows two attempts per test input (pass@2), a rule we formalize in Section 6.

2.2 Program synthesis and refinement loops

Earlier ARC competitions were dominated by hand-written domain-specific languages and search over candidate programs, which is interpretable and exact but limited by the expressiveness of the primitive library. ARC Prize 2025 ran from March to November 2025; 1,455 teams submitted 15,154 entries, the top private-set score was about 24% (NVARC, which builds on the 2024 ARChitects solution), and the Grand Prize remained unclaimed [5, 9]. The report characterizes the common thread as a two-phase refinement

process combining test-time adaptation, data augmentation, and a feedback signal derived from the demonstrations.

2.3 Test-time training

Test-time training (TTT) temporarily updates model parameters at inference time using a loss built from the test task’s own demonstrations, an idea that predates ARC [27]. Akyürek et al. identify three components that make TTT work on ARC: (1) initial fine-tuning on similar tasks, (2) an auxiliary-task format with augmentations (including leave-one-out tasks built from the demonstrations), and (3) per-instance adapter training [2]. With an 8B-parameter model they report 53% accuracy on the ARC-AGI-1 public validation set, nearly 25% above the previous public purely neural state of the art [2]. This is the strongest published evidence that adapter-based adaptation (Section 4.4) can matter for ARC, and it also shows that adaptation *at test time* and *augmentation* are doing the decisive work, not fine-tuning loss alone.

2.4 Small and from-scratch models

Two ARC Prize 2025 paper-award systems show that scale is not the only route. The Tiny Recursive Model (about 7M parameters) reports roughly 45% on ARC-AGI-1 and 8% on ARC-AGI-2 through deep supervised refinement, and CompressARC, an MDL-based per-puzzle system trained without pretraining, reports roughly 20–34% on ARC-AGI-1 and 4% on ARC-AGI-2 [4]. These figures are the developers’ reports and should be read as context, not as baselines reproduced here. They motivate comparing against methods whose compute and parameter budgets are explicit.

3 Dataset and Representation

Represent a grid as

$$G \in \{0, 1, \dots, 9\}^{h \times w},$$

where h and w are its height and width, and each cell stores one of ten color identifiers.

A task supplies demonstrations

$$\mathcal{D} = \{(X_i, Y_i)\}_{i=1}^n,$$

where X_i is an input grid and Y_i is the corresponding output. The intended transformation f should satisfy

$$f(X_i) = Y_i \quad \text{for each supplied demonstration } i.$$

The system must then predict $f(X_*)$ for an unseen input X_* . Let $\mathcal{H}$ be a hypothesis class of candidate transformations and

$$\mathcal{H}_{\mathcal{D}} = \{g \in \mathcal{H} : g(X_i) = Y_i \quad \forall i\} \quad (1)$$

the set consistent with the demonstrations. Several $g \in \mathcal{H}_{\mathcal{D}}$ may disagree on X_* , so fitting the examples does not uniquely identify the underlying rule; the solver needs an inductive bias that selects among them.

3.1 Matrix view of grids and symmetries

Encode G in one-hot form as a matrix $\tilde{G} \in \{0, 1\}^{hw \times 10}$ (row-major order), so that $\tilde{G} \mathbf{1}_{10} = \mathbf{1}_{hw}$. Geometric operations act as permutation matrices on cell positions, and color relabelings act as permutation matrices on the color axis. With the exchange matrix J_k

(ones on the anti-diagonal), a horizontal reflection and a clockwise rotation are

$$G \mapsto G J_w, \quad G \mapsto (J_h G)^T. \quad (2)$$

The eight rotations and reflections form the dihedral group D_4 . A joint geometric–color augmentation (g, π) acts as

$$\tilde{G} \mapsto P_g \tilde{G} \Pi_\pi, \quad P_g^\top P_g = I, \quad \Pi_\pi^\top \Pi_\pi = I, \quad (3)$$

where $P_g \in \{0, 1\}^{hw \times hw}$ and $\Pi_\pi \in \{0, 1\}^{10 \times 10}$ are permutation matrices. Keeping color 0 (background) fixed leaves $|D_4| \cdot 9! = 8 \cdot 362,880 = 2,903,040$ distinct augmentations per task. Applying the *same* (g, π) to every grid of a task preserves the demonstrations’ consistency, but it can break rules that depend on absolute colors or orientation, so augmentation must be validated per method.

3.2 Dataset statistics and exploratory figures

Table 1 lists the counts recorded by the project’s analysis. They do not mean that every task was used for neural training or that the 32 generation cases form the official competition test set.

Table 1: Recorded dataset and experiment counts

Measure	Recorded value
Training tasks	1,000
Training input–output pairs	3,232
Tasks with dimension changes	320
Share with dimension changes	32%
Selected rule-verified tasks	2
Selected rule-verified examples	7
Neural generation evaluation cases	32

Figures 1–4 are the exploratory analyses behind Table 1. Each is explained directly below its plot.

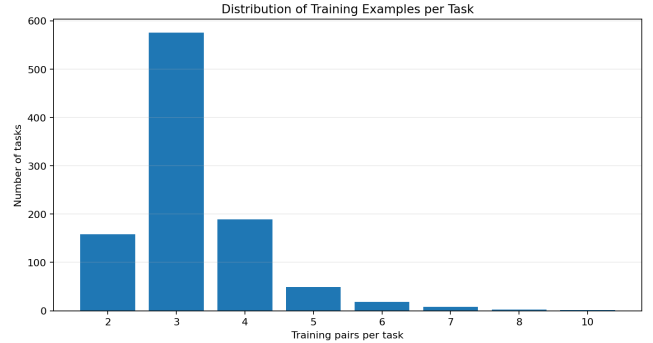


Figure 1: Distribution of training-pair counts: the number of tasks (vertical axis) having a given number of demonstration pairs (horizontal axis).

Figure 1 shows how many demonstrations each task provides. Three pairs is the most common count, followed by four and two; tasks with six or more pairs are rare. The mean is $3,232/1,000 \approx 3.2$ pairs per task, so a solver must infer a rule from roughly three examples, which is why the few-shot setting of Section 2 is demanding.

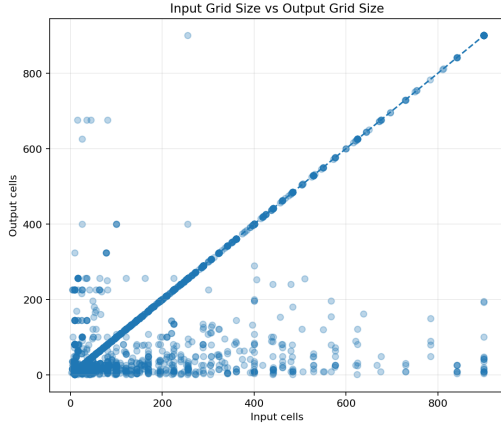


Figure 2: Input grid size versus output grid size, in cells, for each analyzed example. The dashed diagonal marks equal input and output cell counts; the maximum of 900 cells corresponds to a 30×30 grid.

Figure 2 compares input and output sizes. Points on the diagonal keep their size, a large cluster lies below it (outputs smaller than inputs), and a sparser set lies above it (outputs larger). The spread shows that output size cannot be assumed equal to input size, which motivates the dimension analysis that follows.

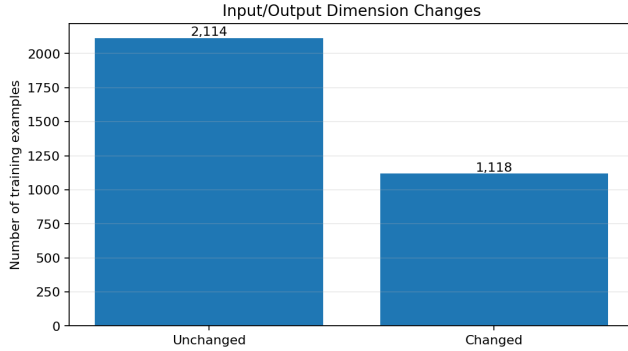


Figure 3: Number of training examples whose output dimensions are unchanged (2,114) or changed (1,118) relative to the input.

Figure 3 counts examples (pairs), not tasks: 2,114 unchanged and 1,118 changed, which sum to the 3,232 pairs in Table 1. About $1,118/3,232 \approx 34.6\%$ of *pairs* change dimensions, while 32% of *tasks* contain at least one such pair; the two percentages use different units and should not be confused. A dimension change is present when at least one example in a task has

$$(h_{\text{in}}, w_{\text{in}}) \neq (h_{\text{out}}, w_{\text{out}}),$$

and the recorded task-level proportion is $320/1000 = 32\%$. This motivates treating output dimensions as part of the prediction problem instead of assuming that every transformation preserves shape.

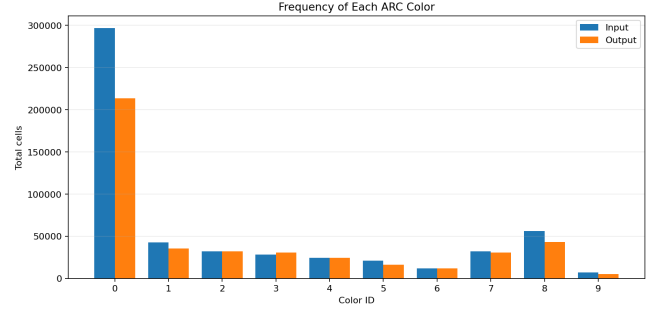


Figure 4: Total cell counts for each color identifier (0–9) across all input grids (blue) and all output grids (orange).

Figure 4 shows that color 0, the background, dominates both inputs and outputs by a wide margin, and that outputs contain fewer background cells than inputs. This is why the augmentation count in Section 3.1 keeps color 0 fixed: a random relabeling of the background would destroy a regularity that most tasks rely on.

4 Experimental Methods

4.1 Deterministic CPU rules

The project implemented explicit candidate transformations for two selected tasks:

- 00576224: alternating horizontal reflection.
- 007bbfb7: pattern-controlled expansion.

Both have a compact matrix form. Writing X for an input grid and J for the exchange matrix, the reflection rule stacks horizontally tiled copies of X and XJ in alternation,

$$Y = \begin{bmatrix} \mathbf{1}_3^\top \otimes X \\ \mathbf{1}_3^\top \otimes XJ \\ \mathbf{1}_3^\top \otimes X \end{bmatrix}, \quad (4)$$

(shown for the 3×3 block layout of the verified examples), and the expansion rule is a masked Kronecker product,

$$Y = M \otimes X, \quad M_{ij} = \mathbf{1}[X_{ij} \neq 0], \quad (5)$$

so block (i, j) of Y is a copy of X where $M_{ij} = 1$ and background elsewhere. The output shape is (h^2, w^2) for a $h \times w$ input, which is one concrete instance of the dimension changes in Figure 3.

The implementation applied each rule to its task’s supplied examples and compared the predicted grids with the known outputs. The recorded result was seven exact reproductions out of seven checked examples (Figures 8 and 9).

This is a useful implementation test. However, because the tasks were selected for verification, it is not an unbiased estimate of accuracy on unseen ARC tasks, and it does not establish that the system can automatically *discover* the correct rule.

4.2 Qwen 3 1.7B

Qwen 3 is a family of pretrained language models with dense and mixture-of-experts variants [22]. The checkpoint recorded for this project was the Qwen 3 1.7B base model. The model directory inspection reported 28 transformer layers, hidden size 2,048, and approximately 3.2 GB of model-weight files (Table 2).

Table 2: Qwen 3 checkpoint details recorded by the notebook

Property	Recorded value
Model family	Qwen 3
Checkpoint	1.7B base
Transformer layers	28
Hidden size	2,048
Weight-file size	Approximately 3.2 GB
Optimizer steps	429 in the recorded run
Initial validation loss	0.6996
Fine-tuned validation loss	0.4407
Exact-match cases	0 / 32

The technical report describes the Qwen 3 family and its training approach [22]. The values in Table 2 come from this project’s checkpoint inspection and experiment record and should not be generalized to every Qwen 3 variant.

Public model cards for the 1.7B checkpoint additionally list 16 query heads and 8 key-value heads with head dimension 128, a feed-forward width of 6,144, a vocabulary of 151,936 tokens, rotary embeddings with base frequency 10^6 , tied input/output embeddings, and a 32,768-token context [18, 23]. Using these published dimensions, the parameter budget can be derived exactly. Per layer, with $d = 2048$, $d_h = 128$, $H = 16$, $H_{kv} = 8$, and $d_{ff} = 6144$:

$$N_{\text{attn}} = d(Hd_h) + 2d(H_{kv}d_h) + (Hd_h)d = 12,582,912, \quad (6)$$

$$N_{\text{ffn}} = 3d d_{ff} = 37,748,736. \quad (7)$$

Over $L = 28$ layers the transformer blocks hold $28(N_{\text{attn}} + N_{\text{ffn}}) \approx 1.41\text{B}$ parameters, and the tied embedding matrix adds $151,936 \times 2048 \approx 0.31\text{B}$, for a total near 1.72B, consistent with the published “1.7B (1.4B non-embedding)” description. Norm weights are negligible and omitted.

4.3 Why transformers matter

Transformers use attention to let representations at different sequence positions interact [28]. A task is serialized as a token sequence $t_1, \dots, t_T$ of cell tokens, row separators, and input/output markers, embedded as $H^{(0)} = E[t] \in \mathbb{R}^{T \times d}$. Each decoder layer applies

$$H' = H + \text{MHA}(\text{RMSNorm}(H)), \quad (8)$$

$$H^+ = H' + \text{FFN}(\text{RMSNorm}(H')), \quad (9)$$

with $\text{RMSNorm}(x) = \frac{x}{\sqrt{\frac{1}{d} \sum_j x_j^2 + \epsilon}} \odot \gamma$ [29]. For query head i , with

$$Q_i = HW_Q^{(i)}, K_i = HW_K^{(\kappa(i))}, V_i = HW_V^{(\kappa(i))},$$

$$\text{head}_i = \text{softmax}\left(\frac{Q_i K_i^\top}{\sqrt{d_h}} + M\right) V_i, \quad (10)$$

where $M_{ts} = -\infty$ for $s > t$ (causal mask) and 0 otherwise. The head outputs are concatenated and multiplied by $W_O \in \mathbb{R}^{Hd_h \times d}$.

Grouped-query attention. In grouped-query attention (GQA) several query heads share one key-value head: $\kappa(i) = \lfloor i H_{kv}/H \rfloor$ maps H query heads onto H_{kv} key-value heads [1]. For Qwen 3 1.7B,

$H/H_{kv} = 2$, so $W_K, W_V \in \mathbb{R}^{2048 \times 1024}$ while $W_Q, W_O \in \mathbb{R}^{2048 \times 2048}$. The key-value cache for T tokens in 16-bit precision then occupies

$$\text{bytes} = 2(K, V) \cdot L \cdot H_{kv} \cdot d_h \cdot T \cdot 2 = 114,688 T, \quad (11)$$

i.e. about 112 KiB per token and roughly 0.875 GiB at $T = 8192$, half of what full multi-head attention would need.

Rotary positions and a 2D caveat. Rotary position embeddings (RoPE) rotate each query and key by an angle proportional to its position m [26]:

$$\tilde{q}_m = R_m q_m, \quad \tilde{k}_n = R_n k_n, \quad \tilde{q}_m^\top \tilde{k}_n = q_m^\top R_{n-m} k_n, \quad (12)$$

with R_m block-diagonal in 2×2 rotations by $m\theta_j$, $\theta_j = b^{-2j/d_h}$, and $b = 10^6$ for Qwen 3. The score therefore depends only on the *one-dimensional* offset $n - m$. In a row-major serialization a vertical neighbor is $w + 1$ tokens away, so the model must learn the 2D adjacency of grid cells rather than receive it from the positional encoding.

Feed-forward block and sequence length. Qwen 3 uses a gated SwiGLU feed-forward network [25],

$$\text{FFN}(x) = W_{\text{down}}(\text{SiLU}(xW_{\text{gate}})) \odot xW_{\text{up}}, \quad (13)$$

and normalizes queries and keys per head (QK-Norm) for stability [22]. If every cell is one token and every row adds one newline token, a task with three 30×30 demonstration pairs and a 30×30 test pair is about $4 \times 2 \times 930 = 7,440$ tokens, so attention cost, which grows as $O(T^2 d)$, is dominated by the grids. This assumes digit-level tokenization, which should be verified for the tokenizer in use.

This flexibility does not guarantee that the model infers the intended transformation. The representation must preserve grid structure, examples must be formatted consistently, and decoding must produce a complete valid grid. A model can learn plausible local token patterns without generating the exact global structure that ARC requires.

4.4 LoRA adapter: purpose, mathematics, and configuration

Full fine-tuning updates all or most pretrained weights. Low-Rank Adaptation (LoRA) freezes the base weights and learns a low-rank update to selected matrices [15]. For a frozen $W_0 \in \mathbb{R}^{d \times k}$,

$$W = W_0 + \frac{\alpha}{r} BA, \quad B \in \mathbb{R}^{d \times r}, A \in \mathbb{R}^{r \times k}, r \ll \min(d, k), \quad (14)$$

so for an input row x the layer computes $y = xW_0 + \frac{\alpha}{r}(xB)A$ without ever forming BA explicitly (here $x \in \mathbb{R}^{1 \times d}$ and the matrix convention matches Eq. (10); some libraries store the transpose). Only A and B receive gradients.

Initialization and scaling. B is initialized to zero and A to small Gaussian noise, so $\Delta W = 0$ at step zero and training starts exactly at the pretrained function [15]. The ratio α/r rescales the update so that changing r does not require retuning the learning rate; a rank-stabilized variant uses $\alpha/\sqrt{r}$ instead [17].

Linear-algebra interpretation. Because $\text{rank}(BA) \leq r$, the update lives in a subspace of dimension at most r per matrix. By the Eckart–Young theorem [12], the best rank- r approximation of any full update ΔW^* is its truncated SVD, so LoRA is effective exactly when the useful task-specific change has a rapidly decaying singular spectrum.

Parameter count. The adapter has $r(d + k)$ trainable parameters instead of dk . As an *illustration only* (the saved adapter configuration is not assumed), Table 3 gives the trainable fraction for Qwen 3 1.7B at rank $r = 16$ under two common choices of target modules, using the dimensions derived in Section 4.2.

Table 3: Illustrative LoRA budget for Qwen 3 1.7B at rank $r = 16$ (derived from published dimensions; not the recorded configuration)

Target modules	Trainable	% of 1.72B
W_Q, W_K, W_V, W_O (all layers)	6.42M	0.37
Attention + $W_{\text{gate}}, W_{\text{up}}, W_{\text{down}}$	17.43M	1.01

For attention only, the per-layer count is $r[(2048+2048) + 2(2048+1024) + (2048+2048)] = 14,336r$, and for the MLP it is $3r(2048+6144) = 24,576r$; both are multiplied by 28 layers. With AdamW, optimizer state costs about 8 bytes per trainable parameter (two FP32 moments), so the adapter state is tens of megabytes rather than the multiple gigabytes of full fine-tuning [15].

Practical variants. QLoRA keeps the frozen base weights in 4-bit precision and trains LoRA adapters on top, reducing memory enough to adapt much larger models on a single GPU [11]. Weight-decomposed LoRA (DoRA) separates magnitude and direction of the update [19]. Both are candidates when memory is the binding constraint.

What should be reported. The adapter rank r , scaling α , dropout, target modules, learning-rate schedule, and trainable-parameter count should be read from the saved adapter configuration and reported as facts; this report does not assume unverified settings. Adapters are modular: a base model can host one adapter per experiment, or, following test-time training [2], one short-lived adapter per task.

The reason to use an adapter is therefore experimental efficiency and modularity, not a guarantee of better reasoning. LoRA can still overfit, learn a poor serialization pattern, or fail to generalize.

5 Alternative Model Choices

5.1 Gemma versus Qwen

Gemma is Google’s family of open-weight models developed using technology and research associated with Gemini. Gemma 3 includes text-only and multimodal sizes, with different context lengths and compute requirements across variants [13, 14]. Qwen 3 also offers multiple sizes and architecture variants, including dense and mixture-of-experts models [22]. Table 4 compares the two families at the level of what this project actually recorded.

Gemma would have been a reasonable alternative if a compatible checkpoint, tokenizer, and training environment had been available. For grid reasoning, a text-only model can suffice if the grid is serialized faithfully. A multimodal model could also be tested with rendered grids as images, but that introduces a different input representation and requires a controlled comparison.

It would be scientifically unsound to state that Gemma is better or worse than Qwen for this task without running both on the same held-out examples, with the same attempt budget, scoring rules, and compute constraints.

5.2 Architectural comparison of candidate families

Modern decoder-only families are close relatives, and their differences are concentrated in normalization placement, attention schedule, and the feed-forward activation [24]. Table 5 summarizes the design choices relevant to a grid-serialization experiment.

Two consequences matter for ARC. First, interleaved local/global attention (Gemma 3) lowers long-context memory but restricts most layers to a window, which may hurt tasks that compare a test grid with demonstrations thousands of tokens earlier; whether it helps or hurts is an empirical question. Second, all four families use RoPE, so the 1D-offset caveat of Section 4.3 applies to each of them unless a 2D-aware positional scheme is added.

5.3 Selection criteria

Model choice should rest on more than general benchmark reputation. For ARC, relevant criteria include:

- exact-grid accuracy on held-out tasks;
- reliable output formatting and grid serialization;
- compatibility with the chosen fine-tuning method (LoRA, QLoRA);
- available model weights and tokenizer, including whether each digit is a single token;
- GPU memory (weights plus KV cache), inference speed, and time;
- license and reproducibility requirements.

No results for these alternative models are claimed in this report.

Table 4: Relevant model-family differences for an ARC experiment

Factor	Qwen 3	Gemma 3
Developer	Qwen team	Google DeepMind
Published variants	Dense and mixture-of-experts models across several sizes	Models from compact text models to larger multimodal variants
Potential ARC use	Serialize demonstrations and generate grid tokens; adapt with a compatible fine-tuning method	Serialize demonstrations and generate grid tokens; suitable variants may also provide image-input capabilities
Compute considerations	Depend on chosen size, precision, context, and architecture	Depend on chosen size, precision, context, and whether image inputs are used
Evidence in this project	A Qwen 3 1.7B base checkpoint was recorded in the notebook	No Gemma run is recorded in the available results
Comparison status	Observed experiment, with the limitations reported here	Candidate for future controlled comparison; no project score available

Table 5: Architectural choices of candidate decoder families (from the cited technical reports and architecture surveys; none of these alternatives was run in this project)

Property	Qwen 3 (dense)	Gemma 3	Llama 3	Mistral 7B
Normalization	RMSNorm (pre-norm)	RMSNorm, additional norm placements	RMSNorm	RMSNorm
Positions	RoPE, base 10^6	RoPE	RoPE	RoPE
Attention	GQA with QK-Norm	GQA with QK-Norm; local sliding-window layers interleaved with global layers (5:1 pattern)	GQA	GQA with sliding-window attention
Feed-forward	SwiGLU	GeGLU	SwiGLU	SwiGLU
Reference	[22]	[13, 24]	[20]	[16]

6 Metrics and Mathematical Definitions

6.1 Cross-entropy loss and perplexity

For target tokens $y_1, \dots, y_T$, the autoregressive cross-entropy objective is

$$\mathcal{L}_{\text{CE}} = -\frac{1}{T} \sum_{t=1}^T \log p_{\theta}(y_t | y_{<t}, x).$$

A lower loss means the model assigned higher average probability to the observed target tokens under the measured objective. It does not imply that every cell in a generated grid is correct. The corresponding per-token perplexity is $\text{PPL} = e^{\mathcal{L}_{\text{CE}}}$.

6.2 Exact-grid accuracy and cell agreement

For prediction $\hat{Y}_i$ and target Y_i , define exact-match accuracy over N examples as

$$A_{\text{exact}} = \frac{1}{N} \sum_{i=1}^N \mathbf{1}[\hat{Y}_i = Y_i].$$

Equality requires matching dimensions and every cell value. When shapes agree, define the Hamming distance $d_H(\hat{Y}, Y) = \sum_{a,b} \mathbf{1}[\hat{Y}_{ab} \neq Y_{ab}]$, which equals $\|\hat{Y} - Y\|_0$, and the cell agreement $a_{\text{cell}} = 1 - d_H/(hw)$. Exact match is the strict event $d_H = 0$ with equal shapes. If cells were correct independently with probability p , exact accuracy would be about p^{hw} ; with $p = 0.99$ this is 0.37 for a 100-cell output and only 0.018 for a 400-cell output.

Independence is an idealization, but it shows why high token-level fit and exact-grid success can diverge sharply.

The recorded Qwen result is $0/32 = 0\%$. The selected CPU-rule result is $7/7 = 100\%$ on the selected demonstrations. The percentages are not directly comparable because the evaluation sets and protocols differ.

6.3 Task-level scoring and validity

The competition instructions define success using the permitted predictions for each task [6]. For a task with two allowed predictions, a task success indicator can be written as

$$S_j = \mathbf{1}[\hat{Y}_{j,1} = Y_j \vee \hat{Y}_{j,2} = Y_j].$$

For M tasks, the task-level score is

$$A_{\text{task}} = \frac{1}{M} \sum_{j=1}^M S_j.$$

This expression describes the two-attempt scoring rule; it is not a calculation of the project’s private leaderboard score. A separate valid-output rate is

$$R_{\text{valid}} = \frac{\text{structurally valid predictions}}{\text{attempted predictions}}.$$

A valid grid may still be wrong. Validity, token loss, exact-match accuracy, and task-level success should therefore be reported separately.

6.4 Augmentation voting

With a set $\mathcal{T}$ of transformations of the form Eq. (3), a model f_θ can be queried on each transformed task and the outputs mapped back by T_a^{-1} . The candidate ranking is by vote count,

$$\hat{Y} = \arg \max_Y \sum_{a \in \mathcal{T}} \mathbf{1}[T_a^{-1}(f_\theta(T_a \mathcal{D}, T_a X_*)) = Y], \quad (15)$$

and the top two distinct candidates are submitted under pass@2. This is the self-consistency component used with test-time training [2]; it is not evaluated in the recorded results here.

6.5 Statistical uncertainty

For k successes in n trials, the two-sided $(1 - \alpha)$ Clopper–Pearson interval is exact [10]. At the extremes it has closed forms: for $k = 0$ the upper limit is $1 - (\alpha/2)^{1/n}$, and for $k = n$ the lower limit is $(\alpha/2)^{1/n}$. Applied to the recorded counts at $\alpha = 0.05$,

$$0/32 : p \in [0, 0.109], \quad (16)$$

$$7/7 : p \in [0.590, 1]. \quad (17)$$

The Qwen run is therefore consistent with a true exact-match rate as high as about 11% on that case distribution, and the seven verified demonstrations are consistent with a rule accuracy as low as about 59%. Neither count supports a strong claim in either direction.

7 Results

7.1 Summary of recorded results

Table 6 places the CPU-rule and Qwen measurements side by side. The two columns use different evaluation scopes, so the table is a record, not a ranking.

Table 6: Results recorded in the current project

Measure	CPU rule verification	Qwen 3 experiment
Evaluation scope	Two selected tasks, seven training demonstrations	32 recorded generation cases
Exact matches	7 / 7	0 / 32
Recorded accuracy	100% on selected demonstrations only	0% on recorded generation cases
Validation loss	Not applicable to deterministic rules	0.6996 before and 0.4407 after fine-tuning
Interpretation	Implemented rules reproduced the checked examples	Loss decreased, but exact-grid success was not observed in these cases
Main caveat	No broad generalization estimate	No official competition score or general impossibility claim

7.2 Neural run: loss and generation quality

The relative reduction in recorded validation loss is

$$\frac{0.6996 - 0.4407}{0.6996} \times 100\% \approx 37.0\%,$$

equivalent to a drop in per-token perplexity from $e^{0.6996} \approx 2.01$ to $e^{0.4407} \approx 1.55$. The recorded loss does not state which tokens it averages over; if it includes the serialized demonstrations as well as the target output, part of the improvement reflects format learning rather than prediction of the output grid. Figures 5–7 document the run.

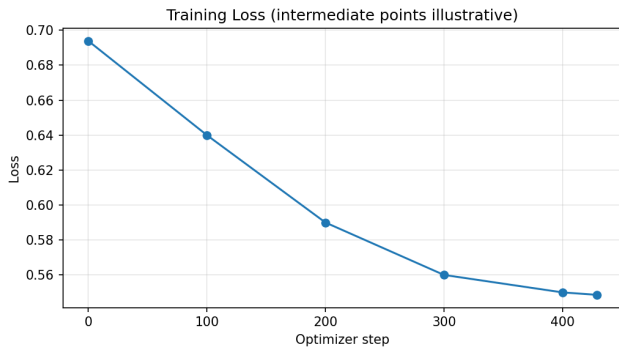


Figure 5: Training loss versus optimizer step for the recorded run. The plot’s own title marks the intermediate points as illustrative.

Figure 5 shows a decreasing curve from about 0.69 at step 0 to about 0.55 near the final step (429 steps, Table 2). Two cautions apply. The plot labels its intermediate points illustrative, so it is a qualitative sketch rather than measured evidence, and the quantitative source is `Tables/training_loss_points.csv`. In addition, the final plotted training value is above the reported validation loss of 0.4407; training and validation losses need not be comparable (different data, averaging window, or regularization), but the difference should be explained before this figure is used quantitatively.

Figure 6 compares the two checkpoints on the same validation measure: loss falls from 0.6996 to 0.4407, the 37.0% reduction computed above. This is the only quantity in the neural run that improved, and it is a token-likelihood measure, not a grid-accuracy measure.

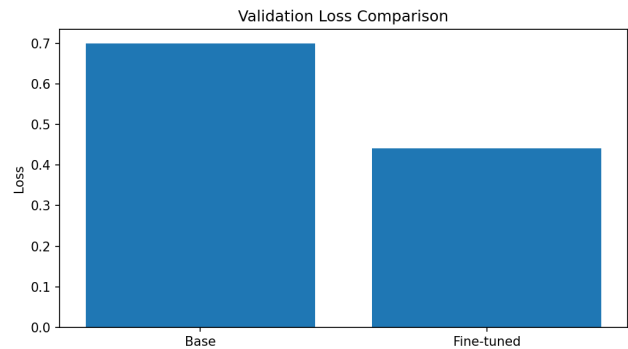


Figure 6: Validation loss of the base checkpoint (0.6996) and the fine-tuned checkpoint (0.4407).

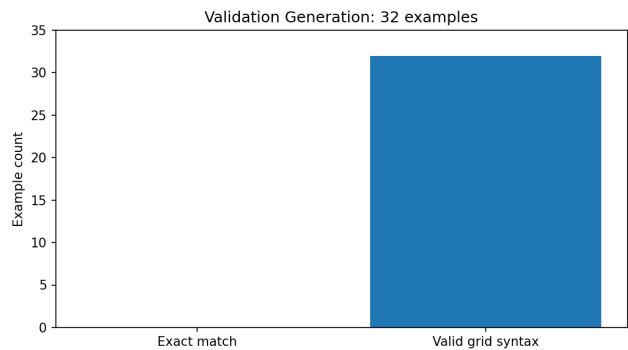


Figure 7: Outcome of the 32-case generation evaluation: zero exact matches and 32 outputs with valid grid syntax.

Figure 7 separates two properties that are easy to conflate. All 32 outputs parse as valid grids, so $R_{\text{valid}} = 32/32 = 100\%$, yet none equals the target (0/32 exact). The fine-tuned model has therefore learned the output *format* but not, in these cases, the output *transformation*. This is the concrete form of the loss/accuracy gap discussed in Section 8; the exact interval on 0/32 is $[0, 0.109]$ (Section 6).

7.3 Rule verification

Figures 8 and 9 document the recorded rule-checking experiment for the two rules of Section 4.1.

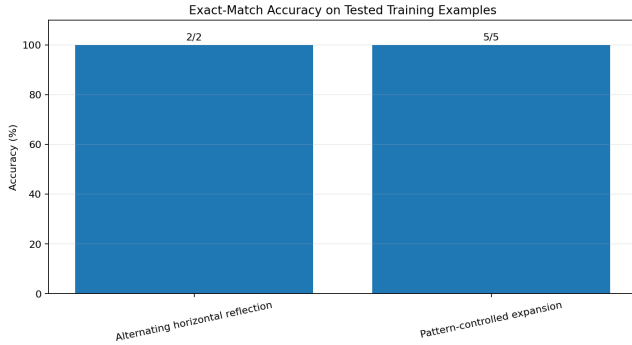


Figure 8: Exact-match accuracy of the two rules on the tested training examples: 2/2 for alternating horizontal reflection and 5/5 for pattern-controlled expansion.

Figure 8 reports 2/2 correct examples for the reflection rule and 5/5 for the expansion rule, i.e. the seven of seven quoted throughout this report. The bars show 100% only because the sample is tiny and was selected for verification.

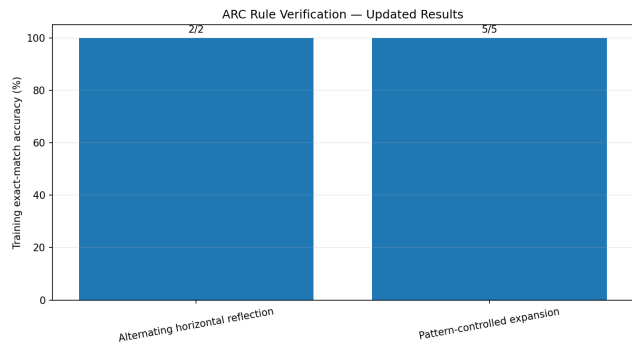


Figure 9: Supplementary rule-verification plot (titled “Updated Results”) with the same per-rule counts, 2/2 and 5/5.

Figure 9 repeats the same per-rule counts in a second, later-titled plot, which confirms that the verification result is stable between the two analysis runs. The 100% value applies only to the seven selected demonstrations and must not be described as competition accuracy; its exact lower confidence limit is about 59% (Section 6).

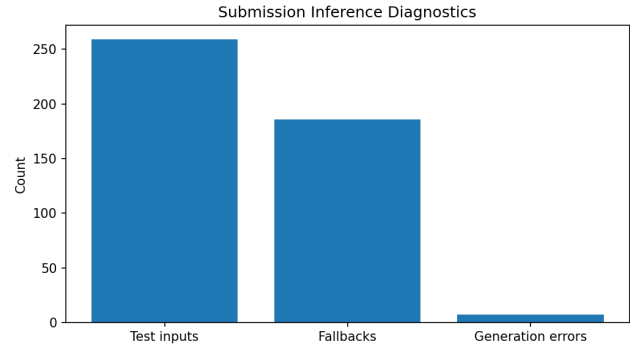


Figure 10: Counts from the submission-inference stage: test inputs processed, inputs that used a fallback prediction, and generation errors.

7.4 Submission diagnostics

Figure 10 summarizes how the test predictions were produced. Reading the bar heights, roughly 260 test inputs were processed, roughly 185 of them (about 70%) used a fallback prediction rather than a model-generated grid, and fewer than 10 raised generation errors; exact counts should be taken from the saved artifacts. The practical consequence is that any score derived from this submission mostly reflects the fallback strategy, not the neural model, so fallback rate should be reported alongside every score.

7.5 Transformation signatures

Figure 11 visualizes per-task transformation features so that tasks with similar signatures (for example, shape changes or color-count changes) can be grouped into candidate rule families, which is the purpose of the candidate rule-family analysis in the retained task tables. The plot is dense; the column names and exact values are in the task-level feature files listed in the appendix, and columns with almost constant color carry little discriminating information.

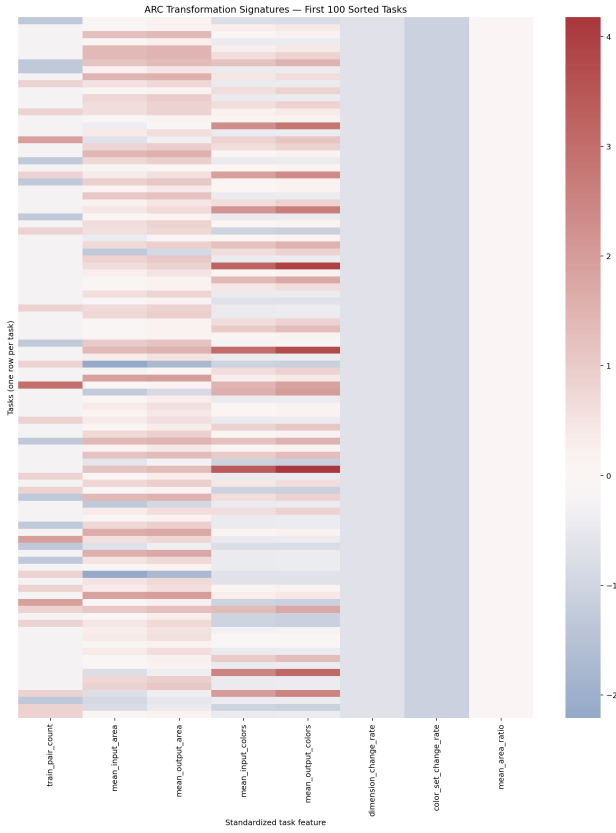


Figure 11: Heatmap of transformation-signature features for 100 tasks, from the dataset analysis; rows are tasks and columns are features, with a diverging color scale.

8 Discussion and Scientific Significance

8.1 What the experiment teaches us

The first lesson is that training loss and task success measure different things. The measured validation loss decreased (Figure 6), but all 32 valid outputs were wrong (Figure 7). The likely causes cannot be determined from these two metrics alone. A follow-up experiment must inspect saved predictions and classify errors such as incorrect shape, wrong cells, malformed serialization, or a transformation inconsistent with the demonstrations.

The second lesson is that deterministic rules are useful interpretable baselines. A rule can be tested against every known example, and failure can be traced to a specific transformation (for instance, a wrong Kronecker mask in Eq. (5)). However, the rule must be discovered and selected correctly; a manually verified implementation is not evidence that rule discovery has been solved.

The third lesson is that model adaptation should be judged on generalization. LoRA makes parameter-efficient experiments feasible, but parameter efficiency is an engineering advantage rather than a direct measure of reasoning. The key question is whether adaptation improves exact accuracy on tasks not used to select the model.

8.2 Why the transformer architecture matters

Attention-based models can relate tokens across a serialized grid and across multiple demonstrations. This offers a flexible mechanism for learning patterns, but does not guarantee systematic abstraction. The serialization must preserve row boundaries and dimensions, and the training objective must align sufficiently with exact output requirements.

This is a central distinction between a language-model objective and a symbolic task objective. Language-model likelihood rewards token probability; ARC exact-match evaluation rewards a complete correct transformation. Future methods may benefit from explicit grid representations, 2D-aware positions, executable candidate programs, constrained decoding, or verification after generation.

8.3 Why compare symbolic and neural systems?

The methods have complementary strengths. Explicit rules are interpretable and easy to verify when applicable. Neural models can propose candidates beyond a hand-written rule library, but can produce plausible yet incorrect outputs. A hybrid system could use a model to propose hypotheses and a deterministic executor to test them against the demonstrations, which is a refinement loop in the sense of [9].

This hybrid is a research hypothesis, not a result of the present experiment. It should be tested by freezing all methods before evaluation and measuring them on the same held-out tasks.

9 Research Questions, Hypotheses, and Protocol

9.1 Research questions

The recorded results and the literature in Section 2 motivate four questions.

- RQ1.** Does LoRA fine-tuning of a small base model improve *exact* held-out accuracy, or only token likelihood?
- RQ2.** How much do augmentation (Eq. (3)) and voting (Eq. (15)) add on top of fine-tuning?
- RQ3.** Does per-task test-time adapter training [2] close the gap that fine-tuning alone leaves?
- RQ4.** Does a hybrid of neural proposal and deterministic verification outperform either component alone?

9.2 Falsifiable hypotheses

Each question maps to a hypothesis with a pre-stated failure condition. H1: LoRA fine-tuning raises held-out exact accuracy above the base model (falsified if the paired difference is not positive at the chosen level). H2: voting over $|\mathcal{T}| \geq 8$ transformations raises pass@2 over a single query (falsified if no gain). H3: per-task TTT exceeds H2 on the same tasks. H4: the hybrid solves a strict superset of the tasks solved by the CPU rules alone.

9.3 Evaluation protocol

To avoid leakage, split by *task*, never by pair: partition the 1,000 training tasks into training and development tasks, select hyperparameters on development tasks only, and report once on the 120 public evaluation tasks. Freeze checkpoints, tokenizer, adapter configuration, seeds, and decoding settings before the final run. Report for every method: exact task-level pass@2, valid-output

rate, attempts per task, fallback rate, runtime, peak memory, and the complete task split.

9.4 Statistical analysis

Because methods are run on the same tasks, outcomes are paired binary vectors. For two methods let b be the number of tasks solved only by the first and c only by the second. Under the null of equal accuracy, the discordant count follows $\text{Binomial}(b + c, \frac{1}{2})$ and McNemar’s statistic is

$$\chi^2 = \frac{(b - c)^2}{b + c} \quad (\text{exact binomial test if } b + c < 25) \quad (18)$$

[21]. Alongside it, report Clopper–Pearson intervals for each method and a task-level bootstrap for the difference. On a 120-task evaluation set one task is worth about 0.83 percentage points (somewhat less when tasks have several test inputs and are scored fractionally), so differences of a few tasks are within noise.

9.5 Threats specific to pretrained models

Qwen 3 was pretrained on roughly 36 trillion tokens [23]; public ARC tasks and discussions may appear in such corpora. The ARC Prize 2025 report discusses this knowledge-dependent overfitting risk [9], so any gain on public tasks should be corroborated on held-out or semi-private sets before it is interpreted as reasoning.

10 Limitations and Threats to Validity

- (1) **Non-comparable evaluations.** The CPU and Qwen results come from different subsets and protocols.
- (2) **Selection bias.** Two selected tasks cannot estimate performance on the full task distribution.
- (3) **Small generation evaluation.** The recorded 32 cases are insufficient to establish broad generalization or an official leaderboard result; the exact interval on 0/32 is $[0, 0.109]$.
- (4) **Illustrative training curve.** Figure 5 labels its intermediate points illustrative, so only the endpoints and the logged data should be treated as measurements.
- (5) **Fallback-dominated submission.** Figure 10 indicates that most test predictions came from a fallback rather than the model.
- (6) **Missing failure examples.** The original Qwen failure outputs were not available in the prior artifact audit. This report therefore does not claim a particular cause for the zero exact matches.
- (7) **Unverified adapter configuration.** The exact LoRA rank and target modules must be read from the saved adapter configuration before they can be reported as facts; Table 3 is illustrative only.
- (8) **No alternative-model experiment.** Gemma, Llama, and Mistral are discussed as candidates, not as evaluated baselines.
- (9) **Possible pretraining contamination.** Overlap between public ARC material and pretraining data cannot be excluded.
- (10) **No novelty claim.** A systematic literature and baseline comparison is still needed.

11 Future Work

The next experiment should use the same held-out tasks for all methods. At minimum, it should compare:

- (1) deterministic CPU rules;
- (2) Qwen 3 with a fixed checkpoint and decoding configuration, with and without LoRA;
- (3) Gemma, if a compatible checkpoint is available;
- (4) a hybrid candidate-generation and verification method.

For every method, report exact task-level accuracy, valid-output rate, fallback rate, number of attempts, runtime, memory use, and the complete task split. The model checkpoint, tokenizer, adapter configuration, random seed, and decoding settings should be saved with the predictions.

An ablation study should remove one component at a time, such as augmentation, test-time adaptation, or voting, to establish which components contribute. A LoRA sweep over rank $r \in \{4, 8, 16, 32\}$ and over attention-only versus attention-plus-MLP targets would show how much capacity the task needs, and a comparison of 1D and 2D-aware positional schemes would test the caveat in Section 4.3. All reported gains should be measured on the same held-out set with the analysis of Section 9.

12 Six-Criterion Research Assessment

Table 7 lists the current evidence against each of the six research criteria; the paragraph after it interprets each row.

12.1 Reading the assessment

Accuracy. The only exact-match evidence is 7/7 on selected demonstrations and 0/32 on generation cases; their intervals (Section 6) are wide and overlap nothing comparable. **Universality.** Matrix-form rules such as Eq. (5) are exact but task-specific; universality needs a discovery mechanism, not more hand-written rules. **Progress.** The measurable progress so far is diagnostic: a 37.0% loss reduction that did not translate into exact outputs, which redirects effort toward decoding, augmentation, and verification. **Theory.** The linear-algebraic account supplies testable statements (rank- r updates, 1D RoPE offsets, voting over a group action) that the protocol of Section 9 can falsify. **Completeness.** Raw artifacts are retained in the project tree and indexed in Appendix A. **Novelty.** Test-time training, augmentation voting, and LoRA adaptation on ARC are published [2, 9]; any novelty would have to come from a specific, measured combination.

Table 7: Current evidence against the six research criteria

Criterion	Current evidence	Required next step
Accuracy	Seven selected rule examples passed; Qwen recorded zero exact matches in 32 generation cases.	Run a controlled held-out comparison.
Universality	No independent cross-benchmark evidence.	Test separate tasks or benchmarks without leakage.
Progress	The work establishes preliminary baselines and exposes a loss/accuracy gap.	Demonstrate reproducible improvements toward the target.
Theory	The report defines the task, loss, LoRA update, and scoring metrics.	Test competing hypotheses and explain why rules generalize.
Completeness	Core analysis tables, predictions, and figures are retained.	Verify the complete test submission and official schema.
Novelty	No novelty claim is established by the current results.	Compare against public ARC systems and relevant literature.

13 Conclusion

This project recorded 1,000 training tasks and 3,232 training pairs, found dimension changes in 320 tasks, and verified two deterministic rules against seven selected demonstrations. A separate Qwen 3 experiment reduced validation loss from 0.6996 to 0.4407, but produced 32 valid grids with zero exact matches in its 32-case generation evaluation.

The evidence is preliminary. It neither proves that explicit rules outperform neural models nor shows that Qwen 3 cannot solve ARC. Instead, it demonstrates why training loss, selected example checks, and held-out task success must be reported separately.

The scientific priority is a controlled evaluation of CPU rules, Qwen 3 with and without LoRA, an available Gemma checkpoint, and a hybrid method, with augmentation voting and test-time adaptation as explicit ablations. A credible claim of progress requires reproducible gains on the same held-out tasks, correct output formatting, and transparent reporting of compute, fallback rate, and failure cases.

A Artifact Inventory

The files in Table 8 remain available in the project tree. They are not expanded into raw row-by-row appendix tables, which previously caused the report to exceed 100 pages.

Table 8: Project data artifacts

File	Purpose
Tables/archive_manifest.csv	Archive inventory.
Tables/color_frequency.csv	Color-frequency counts.
Tables/cpu_prediction_integrity_audit.csv	Structural checks on saved CPU predictions.
Tables/dimension_change_counts.csv	Dimension-change summary.
Tables/eda_summary.csv	Exploratory analysis summary.
Tables/example_level_eda.csv	Example-level EDA.
Tables/example_level_features.csv	Example-level features.
Tables/metrics.csv	Recorded experiment metrics.
Tables/rule_test_predictions.json	Saved rule predictions.
Tables/rule_verification_results.csv	Selected rule-check results.
Tables/task_level_features.csv	Task-level features.
Tables/task_rule_analysis.csv	Candidate rule-family analysis.
Tables/task_summary.csv	Task-level summary.
Tables/training_loss_points.csv	Training-loss plot data.

References

- [1] Joshua Ainslie, James Lee-Thorp, Michiel de Jong, Yury Zemlyanskiy, Federico Lebrón, and Sumit Sanghai. 2023. GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints. In *Proceedings of EMNLP*. <https://arxiv.org/abs/2305.13245>.
- [2] Ekin Akyürek, Mehul Damani, Linlu Qiu, Han Guo, Yoon Kim, and Jacob Andreas. 2024. The Surprising Effectiveness of Test-Time Training for Abstract Reasoning. *arXiv:2411.07279*. <https://arxiv.org/abs/2411.07279>.
- [3] ARC Prize. ARC-AGI-2 dataset repository. <https://github.com/arcprize/ARC-AGI-2>.
- [4] ARC Prize. 2025. ARC Prize 2025: winners and paper awards. <https://arcprize.org/competitions/2025>.
- [5] ARC Prize. 2025. ARC Prize 2025 Results & Analysis. <https://arcprize.org/blog/arc-prize-2025-results-analysis>.
- [6] ARC Prize. 2026. ARC Prize 2026: ARC-AGI-2 Competition. <https://www.kaggle.com/competitions/arc-prize-2026-arc-agi-2>.
- [7] François Chollet. 2019. On the Measure of Intelligence. *arXiv:1911.01547*. <https://arxiv.org/abs/1911.01547>.
- [8] François Chollet et al. 2025. ARC-AGI-2: A New Challenge for Frontier AI Reasoning Systems. *arXiv:2505.11831*. <https://arxiv.org/abs/2505.11831>.
- [9] François Chollet, Mike Knoop, Gregory Kamradt, and Bryan Landers. 2026. ARC Prize 2025: Technical Report. *arXiv:2601.10904*. <https://arxiv.org/abs/2601.10904>.
- [10] C. J. Clopper and E. S. Pearson. 1934. The use of confidence or fiducial limits illustrated in the case of the binomial. *Biometrika* 26(4):404–413.
- [11] Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. 2023. QLoRA: Efficient Finetuning of Quantized LLMs. *arXiv:2305.14314*. <https://arxiv.org/abs/2305.14314>.
- [12] Carl Eckart and Gale Young. 1936. The approximation of one matrix by another of lower rank. *Psychometrika* 1(3):211–218.
- [13] Gemma Team. 2025. Gemma 3 Technical Report. <https://arxiv.org/abs/2503.19786>.
- [14] Google DeepMind. Gemma 3 Model Card. https://ai.google.dev/gemma/docs/core/model_card_3.
- [15] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. 2022. LoRA: Low-Rank Adaptation of Large Language Models. In *International Conference on Learning Representations*. <https://arxiv.org/abs/2106.09685>.
- [16] Albert Q. Jiang et al. 2023. Mistral 7B. *arXiv:2310.06825*. <https://arxiv.org/abs/2310.06825>.
- [17] Damjan Kalajdzievski. 2023. A Rank Stabilization Scaling Factor for Fine-Tuning with LoRA. *arXiv:2312.03732*. <https://arxiv.org/abs/2312.03732>.
- [18] KerasHub. qwen3_1.7b_en model configuration. https://huggingface.co/keras/qwen3_1.7b_en.
- [19] Shih-Yang Liu et al. 2024. DoRA: Weight-Decomposed Low-Rank Adaptation. *arXiv:2402.09353*. <https://arxiv.org/abs/2402.09353>.
- [20] Llama Team, AI @ Meta. 2024. The Llama 3 Herd of Models. *arXiv:2407.21783*. <https://arxiv.org/abs/2407.21783>.
- [21] Quinn McNemar. 1947. Note on the sampling error of the difference between correlated proportions or percentages. *Psychometrika* 12(2):153–157.
- [22] Qwen Team. 2025. Qwen3 Technical Report. *arXiv:2505.09388*. <https://arxiv.org/abs/2505.09388>.
- [23] Qwen Team. Qwen3-1.7B-Base model card. <https://huggingface.co/Qwen/Qwen3-1.7B-Base>.
- [24] Sebastian Raschka. GPT, Llama, Qwen, and Gemma architecture comparison. <https://sebastianraschka.com/faq/docs/gpt-llama-qwen-gemma-comparison.html>.
- [25] Noam Shazeer. 2020. GLU Variants Improve Transformer. *arXiv:2002.05202*. <https://arxiv.org/abs/2002.05202>.
- [26] Jianlin Su, Yu Lu, Shengfeng Pan, Ahmed Murtadha, Bo Wen, and Yunfeng Liu. 2021. RoFormer: Enhanced Transformer with Rotary Position Embedding. *arXiv:2104.09864*. <https://arxiv.org/abs/2104.09864>.
- [27] Yu Sun, Xiaolong Wang, Zhuang Liu, John Miller, Alexei Efros, and Moritz Hardt. 2020. Test-Time Training with Self-Supervision for Generalization under Distribution Shifts. In *International Conference on Machine Learning*. <https://arxiv.org/abs/1909.13231>.
- [28] Ashish Vaswani et al. 2017. Attention Is All You Need. In *Advances in Neural Information Processing Systems*. <https://arxiv.org/abs/1706.03762>.
- [29] Biao Zhang and Rico Sennrich. 2019. Root Mean Square Layer Normalization. In *Advances in Neural Information Processing Systems*. <https://arxiv.org/abs/1910.07467>.